Adaptive Query Execution or Salting? A Cost Model and Decision Procedure for Skewed Joins in Apache Spark

Aditya Malviya
Independent Researcher
Mumbai, India
adityamalviya04692@gmail.com

***Abstract*—Data skew remains the dominant cause of straggler tasks in distributed joins. Apache Spark ships an automatic remedy, the skew-join rule of Adaptive Query Execution (AQE), yet practitioners continue to apply *salting* by hand, and the two are routinely combined without any account of whether the combination helps. In the venues we searched we found no independent, controlled evaluation of Spark's built-in skew-join optimisation, and no treatment of salting under that name: the technique is transmitted entirely through vendor documentation and practitioner blogs, none of which prices its cost. That cost is real. Salting a key across k sub-keys relieves the straggler in proportion to $1/k$ but replicates the matching rows on the opposite side in proportion to k, so total time is convex in k and has an interior minimum that the folklore advice to "pick a reasonable k" cannot locate. We contribute three things. First, an analytical cost model $T(k) = aH/k + bP(k-1) + c$ with the closed-form optimum $k^* = \sqrt{aH/(bP)}$, where H and P are the hot-key row counts on the two sides, together with two ceilings that bound the useful range: a *saturation* bound $k \leq \rho$, where ρ is the hot key's rows over the mean rows per partition, beyond which further splitting relieves nothing, and a *parallelism* bound $k \leq C$. Second, a decision procedure that predicts from data statistics alone whether AQE will suffice, keying on the one-sided/two-sided distinction that determines whether AQE's split-and-replicate degenerates. Third, SKEWBENCH, an open benchmark harness that generates Zipfian-skewed workloads with an aerospace engine-telemetry schema, executes six join strategies, and reports hardware-independent metrics recovered from the Spark event log. Measuring 45 configurations on a 2 000 000-row join, we find that enabling AQE produced no improvement distinguishable from measurement noise at any probe-side thickness (1.01× and 1.02× against measurement floors of 0.1% and 6.5%), and left the critical path unchanged: the longest join-stage task went from 17.0 s to 17.5 s. Inspecting final adaptive plans shows why — on a join whose longest task ran two orders of magnitude beyond the median, Spark's skew-join rule *never fired*. Neither documented trigger knob was responsible: sweeping both to their most permissive settings changed nothing, while lowering `advisoryPartitionSizeInBytes` below the hot partition engaged the rule immediately, since that is the size the rule splits into. A paired control with partition coalescing disabled confirms that AQE's small residual benefit is coalescing, not skew handling. Selective salting, by contrast, cut the critical path 1.9×. Measured in shuffle records the model's replication term is confirmed exactly — selective salting adds 11 rows per salt value against uniform salting's 20000, a ratio of 1818 matching the predicted M/P at $R^2 = 1.000$ — but we decline to claim the closed form for k^* is validated, because the replication coefficient is statistically unidentified in all 3 fits. We report that negative result rather than a fitted curve, and release the cluster configuration under which it becomes testable.**

***Index Terms*—Apache Spark, data skew, adaptive query execution, salting, cost model, distributed joins, lakehouse, benchmark**

I. Introduction

A join between a large fact table and a smaller dimension table is the most common heavy operation in an analytics pipeline, and skew in the join key is the most common reason such a join is slow. When one key carries a large share of the rows, hash partitioning sends them all to a single reducer, one task runs far longer than its peers, and the stage finishes when that task finishes. Adding compute does not help: the straggler is serial. The effect was characterised for parallel joins three decades ago [1], [2] and re-confirmed at production scale in every generation of big-data system since [3]–[5].

Apache Spark [6], [7] addresses skew automatically. Since version 3.0, the *Adaptive Query Execution* (AQE) framework re-optimises a plan between stages using statistics observed at runtime, and its `OptimizeSkewedJoin` rule detects a shuffle partition that is disproportionately large, splits it, and replicates the matching partition on the other side so that every split finds its partners [8]–[10].

And yet practitioners keep salting by hand. *Salting* appends a random suffix drawn from $\{0, \ldots, k-1\}$ to the join key on the skewed side and replicates the matching rows k times on the other side, so that one heavy key becomes k lighter ones. The technique is ubiquitous in industry and, as we establish in Section II, entirely absent from the peer-reviewed literature under that name. It is transmitted through vendor documentation, conference talks and blog posts, none of which prices what it costs.

That silence matters, because salting is not free. Every additional salt value multiplies the rows that must be replicated and shuffled on the opposite side. The straggler term falls as $1/k$; the replication term rises linearly in k. Total time is therefore *convex* in k with an interior minimum, and the standard advice — "pick a reasonable k, say 16 or 32" — is not advice at all. It names a point on a curve whose shape nobody has published.

A. The questions this paper answers

1) Given a join's key distribution and Spark's configuration, will AQE's skew-join rule suffice on its own, or is explicit salting still required?
2) If salting is required, what value of k minimises total cost, and can it be derived from statistics available before the query runs rather than found by trial and error?
3) Does combining AQE with salting help, hurt, or do nothing?

B. Contributions

- **A cost model for salt cardinality.** We formulate the salting trade-off as $T(k) = aH/k + bP(k-1) + c$, where H and P are the hot-key row counts on the skewed and probe sides, and derive the closed-form optimum $k^* = \sqrt{aH/(bP)}$. The single free ratio $\gamma = a/b$ is a property of the engine and hardware, calibrated once and reused (Section IV).
- **Two ceilings that bound the useful range.** Beyond $k = \rho$, where ρ is the hot key's rows divided by the mean rows per partition, the hot task is no longer the straggler and further splitting relieves nothing; beyond $k = C$, the number of concurrently schedulable task slots, the sub-partitions queue rather than overlap. *Salting beyond* $\min(\rho, C)$ *is always waste* — a bound that follows directly from the model and that we could not find stated anywhere in the practitioner literature.
- **A decision procedure.** A rule that predicts, from data statistics and Spark configuration alone, whether AQE will trigger, whether it will suffice, and what k to use otherwise. Its hinge is the distinction between one-sided skew, which AQE is designed for, and two-sided skew, in which split-and-replicate degenerates because each split inherits a large partner (Section V).
- **SKEWBENCH, an open harness.** A reproducible benchmark that generates Zipfian-skewed workloads with an aerospace engine-telemetry schema, executes six join strategies, and recovers hardware-independent metrics — shuffle bytes, spill bytes, and the within-stage ratio of maximum to median task duration — by parsing the Spark event log. It runs on a commodity laptop (Section VI).
- **An empirical evaluation** over 48 configurations, which confirms the decay of AQE's benefit as the probe side thickens, identifies the precondition that actually blocked Spark's skew-join rule from ever firing, and quantifies the replication-cost gap between the two salting variants. It also reports, as a negative result, that the convexity of $T(k)$ does not resolve at single-node scale for selective salting (Section VII).

C. Scope, stated plainly

This is an evaluation and decision-procedure paper, not an algorithm paper. We propose no new partitioner; the space of skew-resilient partitioning algorithms is already well populated [11]–[15]. Our claim is that the prior question — *which of the mechanisms that already ship should a practitioner use, and how should it be parameterised* — has gone unanswered, and that it is answerable.

All measurements were taken on a single node. We are explicit about what that does and does not license. Absolute wall-clock times do not transfer to a cluster and we make no claim that they do; shuffle volume, spill volume and within-stage task-time ratios are properties of the physical plan and the data partitioning rather than of the machine, and it is on those that the quantitative claims rest. Section VIII treats this at length.

II. Background and Related Work

A. Skew in parallel joins

Skew in parallel joins is old and well understood in the abstract. Walton et al. [1] gave a taxonomy and performance model of skew effects; DeWitt et al. [2] proposed practical handling including range partitioning with subset replication for heavy keys — the direct ancestor of what practitioners now call salting. In the MapReduce era, SkewTune [5], [16] repartitioned a straggler's remaining input at runtime, and Mantri [4] established at production scale that outlier tasks dominate job latency.

A substantial line of work proposes new partitioners. For Spark specifically: intermediate-data partitioning for skew mitigation [15] and skew-resilient adaptive parallel processing [14]. In the MapReduce lineage from which these descend: sampling-based and cost-aware partitioning schemes [12], [13]. Irandoost et al. [11] survey the MapReduce field systematically. Their review discusses neither Adaptive Query Execution nor salting — fairly enough for AQE, which shipped in Spark 3.0 after that review was written, but the omission is indicative: the literature has been optimising the mechanism while the question of *selecting* among deployed mechanisms went unasked.

Metwally's work on scaling and load-balancing equi-joins [17] is the closest formal treatment of the idea salting embodies. AM-Join combines randomised key expansion with replication on the opposite side and analyses doubly-skewed keys as the hard case. It is the right formal ancestor to cite, and it is not a study of the technique practitioners actually run.

B. Adaptive Query Execution

AQE re-optimises between stages using runtime statistics: coalescing post-shuffle partitions, converting sort-merge joins to broadcast joins when the observed side turns out small, and splitting skewed partitions [8], [9]. Xue et al. [10] describe the production descendant of this machinery at lakehouse scale and state its skew handling explicitly: the rule discovers skew on the join keys of a shuffled hash join, "which manifests as a few partitions containing significantly more data than others," and eliminates it by "logically splitting those large, consumer-side partitions into smaller, more balanced partitions."

That paper is peer-reviewed, and it is the authoritative account of the mechanism. It is also a systems-description paper written by the vendor and evaluated on the vendor's

engine. What it does not provide — and what does not exist elsewhere — is an independent, controlled evaluation of the open-source rule against the alternative practitioners actually reach for.

Adjacent work confirms the shape of the gap rather than filling it. Learned and adaptive Spark optimisers — AQORA [18], LOCAT [19], fine-grained parameter tuning [20], zero-execution configuration tuning [21] — address join ordering, broadcast selection and configuration knobs. AQORA operates *inside* the AQE framework and does not address skew. Zhang et al. [22] evaluate adaptive query processing rigorously, but on single-node relational engines rather than Spark.

C. The specific absence

We searched dblp, the arXiv full-metadata API, Crossref, and domain-scoped searches over the ACM Digital Library, IEEE Xplore, the VLDB proceedings, Springer, ScienceDirect and MDPI. Two negative results are load-bearing for this paper, and we state them with their limits.

No independent evaluation of Spark's skew-join rule. We found no peer-reviewed paper whose contribution is an evaluation of `spark.sql.adaptive.skewJoin`. The nearest comparative study, Phan et al. [23], compares hash-based, range-based and multi-dimensional range partitioning on MapReduce and Spark — and does not consider AQE. The mechanism is otherwise documented only in vendor material [8], [9] and described from the provider's perspective [10].

No peer-reviewed treatment of salting. A full-metadata arXiv search for `abs:salting AND abs:join` returns five papers, all in condensed-matter physics and astronomy. Searches for "key salting" and "salted join" across the major publisher databases return only unrelated hash-join work. Every source describing the technique as practitioners apply it is a blog post, a tutorial, or vendor documentation. Its formal analogue is randomised key expansion with replication [2], [17], never under that name and never as the object of study.

We note the limits of this claim. Semantic Scholar and OpenAlex were unavailable during our search, so recall rests on dblp, arXiv, Crossref and domain-scoped web search, and dblp's coverage of some journals is imperfect — Phan et al. is itself not indexed there. We therefore claim these absences as strong evidence rather than as proof.

D. Cost models and benchmarking practice

Cost modelling for big-data query processing is well developed [24]–[26], including for shuffle specifically [27], but we are not aware of a cost model for the salting decision. Methodologically we follow LST-Bench [28] in building the benchmark from composable, declared workload patterns, and Nurdin et al. [29] in treating runtime variance as a first-class threat: they measure roughly twofold run-to-run variation for identical lakehouse queries on shared cloud compute, which is precisely why our timed measurements are taken on dedicated local compute and our transferable claims rest on deterministic counters.

III. Problem Formulation

A. Notation

Consider an inner equi-join $F \bowtie_\kappa D$ on key κ, where F is the larger, skewed side with N rows and D the probe side with M rows. Spark hash-partitions both sides into p shuffle partitions (`spark.sql.shuffle.partitions`) and executes a sort-merge join with C concurrently schedulable task slots.

Let the key frequencies on F follow a truncated Zipf law over n distinct keys, $\Pr[\kappa = i] \propto i^{-\theta}$, and let key 1 be the hot key. Write

- H for the rows of F carrying the hot key,
- P for the rows of D carrying the hot key.

Both are obtainable before execution, from table statistics, a sampled frequency sketch, or the catalogue.

It is convenient to express P relative to the probe side's mean rows per key, $\bar{m} = M/n$, through a *hot-key multiplier*

$$\mu \;=\; \frac{P}{\bar{m}}, \tag{1}$$

so that $\mu = 1$ is a probe side with no concentration at all — one-sided skew — and larger μ makes the skew progressively two-sided. Treating the degree of two-sidedness as continuous rather than binary matters for two reasons, one practical and one methodological, and we return to both in Section VI.

B. The skew ratio

The quantity that governs everything is dimensionless:

$$\rho \;=\; \frac{H}{N/p} \tag{2}$$

the hot key's rows divided by the mean rows per shuffle partition. It says how many ordinary partitions' worth of work has been concentrated into one.

Equation (2) is a property of the data and the partition count alone. It contains no hardware term, which is what allows a rule built on it to be stated once and applied on any cluster. When $\rho \le 1$ there is no straggler to remove. When $\rho \gg 1$ the stage's completion time is essentially the hot task's completion time, regardless of how many executors are provisioned.

C. Why μ cannot be taken to its limit

There is a natural temptation to construct the two-sided case by drawing both sides from the same Zipf law. It does not work, and the reason is worth stating because it constrains any experiment in this area.

If both sides follow the same law, the hot key contributes $H \times P$ rows to the join output on its own. At the scales used here that is order 10^9 rows from a single key, a thousandfold amplification of the fact table — the join has become a near-cartesian product, and any measurement of it reports the cost of materialising that product rather than the cost of the skew mitigation under test. Bounding μ explicitly keeps the output tractable and, more usefully, promotes P to an independent variable. Equation (4) predicts that the optimal salt cardinality falls as $1/\sqrt{P}$; that prediction is only testable if P can be swept.

D. Two mechanisms, one problem

1) AQE skew-join: Spark's `OptimizeSkewedJoin` rule classifies a shuffle partition as skewed when its size exceeds *both* an absolute threshold (`skewedPartitionThresholdInBytes`) and a multiple (`skewedPartitionFactor`) of the median partition size. It then splits the offending partition into sub-partitions of roughly the advisory size and *replicates the corresponding partition of the other side* to each split, so that every sub-partition retains its join partners [9], [10].

The replication is the hinge. If the other side's matching partition is small — the one-sided case — replication is nearly free and the rule works as advertised. If the other side is also concentrated on the same key — the two-sided case — then each of the s splits inherits a large partner, the probe-side work multiplies by s, and the relief on the critical path is partly or wholly cancelled. Metwally [17] identifies doubly-skewed keys as the hard case for exactly this reason.

2) Salting: Salting rewrites the key. On F, the hot key is replaced by (κ, σ) with σ drawn uniformly from $\{0, \ldots, k-1\}$; on D, each matching row is emitted k times, once per salt value. One heavy key becomes k lighter ones.

We distinguish two variants, because their costs differ by orders of magnitude and the literature does not separate them:

- Uniform salting: every row of D is replicated k times. This is the form shown in most tutorials. Replication cost scales with M.
- Selective salting: only rows whose key is hot are replicated; all other rows carry salt 0 on both sides and join exactly as before. Replication cost scales with P.

Since $P \ll M$ in any realistic fact–dimension join, the distinction is not a detail. We evaluate both.

E. What is actually being traded

Salting does not remove work; it moves it. Splitting the hot key across k sub-keys reduces the critical task's input from H to approximately H/k rows, and simultaneously introduces $P(k-1)$ probe-side rows that did not previously exist. One term falls, the other rises, and the practitioner is choosing a point on that curve without being told there is a curve.

The next section prices it.

IV. A Cost Model for Salt Cardinality

A. Formulation

Under selective salting with cardinality k, total stage time decomposes into three parts.

The critical task. The hot key's H rows are spread over k sub-partitions, so the heaviest task processes approximately H/k rows. Write a for the marginal cost of a row on the critical path.

Replication. The P matching probe-side rows are emitted k times instead of once, adding $P(k-1)$ rows to shuffle, sort and merge. Write b for the marginal cost of a replicated row.

Everything else. Scanning both inputs, joining the cold keys, and the terminal aggregation are all independent of k. Collect them into c.

$$T(k) \;=\; a\,\frac{H}{k} \;+\; b\,P\,(k-1) \;+\; c, \qquad k \geq 1. \tag{3}$$

B. The optimum

T is the sum of a convex decreasing term and a linear increasing term, hence strictly convex on $k > 0$, with a unique interior minimum. Differentiating,

$$\frac{dT}{dk} \;=\; -\frac{aH}{k^2} + bP \;=\; 0 \qquad \Longrightarrow \qquad \boxed{k^* \;=\; \sqrt{\frac{aH}{bP}} \;=\; \sqrt{\gamma\,\frac{H}{P}}} \tag{4}$$

where $\gamma = a/b$ is the ratio of the two marginal costs. Four properties of Equation (4) are worth drawing out.

First, **it depends on the workload only through** H/P, the ratio of hot-key rows on the two sides. That ratio is cheap to estimate and is exactly what a frequency sketch on the join key already yields.

Second, γ **ought to be a property of the engine and the hardware rather than of the query**: row-processing cost on the critical path and replicated-row shuffle cost both scale with the machine, so their ratio should be more stable than either alone, and one calibration should serve many workloads. We state this as a hypothesis rather than a result because our own measurements do not support it — see Section VII, where fitted γ varies by $121\times$ across three workloads on one machine. That instability is diagnostic of the fits being dominated by the $k = 1$ to $k = 2$ step rather than by the replication term, and it is the second reason, alongside the unresolved convexity, that we do not claim the closed form is validated.

Third, k^* **falls as** $1/\sqrt{P}$. Thickening the probe side's hot key makes every additional salt value more expensive while leaving straggler relief unchanged, so the optimum retreats. This is the model's sharpest falsifiable prediction, and Section VII tests it directly by sweeping μ over a factor of 32.

Fourth, and least intuitively, k^* **grows only as the square root of the skew**. Doubling the hot key's dominance does not double the salt cardinality; it multiplies it by $\sqrt{2}$. This is the structural reason aggressive salting is so often counterproductive: intuition says respond to severe skew with large k, and the mathematics says respond with a modestly larger one.

C. Two ceilings

Equation (4) is unconstrained, and taken literally it will sometimes recommend a k that buys nothing at all.

1) Saturation: Once the hot partition has been divided below the size of an ordinary partition, the hot task stops being the straggler — some other partition now finishes last — and further division relieves nothing while continuing to pay replication. From Equation (2), that point is

$$k_{\text{sat}} \;=\; \rho \;=\; \frac{H}{N/p}. \tag{5}$$

Salting beyond ρ **is always waste.** The bound is immediate from the model, requires no calibration, and we have not found it stated in the practitioner literature.

2) Parallelism: Sub-partitions beyond the number of concurrently schedulable task slots C queue rather than overlap, so the straggler term stops falling at $k = C$ while the replication term keeps rising:

$$k_{\text{par}} \;=\; C. \tag{6}$$

3) The operational rule:

$$k_{\text{eff}} \;=\; \max\Big(1,\; \big\lfloor \min(k^*,\, \rho,\, C) \big\rceil\Big). \tag{7}$$

Which of the three binds is itself diagnostic. If k^* binds, the workload is replication-limited and the probe side is the thing to shrink. If ρ binds, the skew is mild relative to the partition count and raising p would serve better than salting. If C binds, the cluster is the limit and salting further is pure overhead — the common case on small clusters, and the one in which tutorial defaults do the most damage.

D. Uniform salting

For uniform salting the replication term scales with the whole probe table, so Equation (3) becomes $T_u(k) = aH/k + bM(k-1) + c$ and $k_u^* = \sqrt{\gamma H/M}$. Since $M \gg P$, the optimum is correspondingly smaller — often below 2, meaning uniform salting is frequently not worth doing at all. This falls out of the model rather than having to be discovered empirically, and it is a good illustration of why pricing the technique is worth the trouble.

E. Calibration

Given measurements $\{(k_i, T_i)\}$, Equation (3) is *linear* in (a, b, c) under the basis $[H/k,\; P(k-1),\; 1]$. Fitting is therefore ordinary linear least squares: no initial guess, no local minima, no optimiser hyperparameters, and a fit that is a deterministic function of the measurements. Three distinct values of k suffice to identify three coefficients; we use seven.

V. The Decision Procedure

Section IV answers "how much salt." The prior question is whether to salt at all, given that AQE may already handle the case. The procedure in Algorithm 1 answers both, from statistics available before execution.

The procedure has three branches, and each corresponds to a distinct failure or success mode.

A. Branch 1: no straggler

When $\rho < \rho_{\min}$ (we use $\rho_{\min} = 2$) the hot key amounts to less than a couple of ordinary partitions. There is nothing to relieve, and both mechanisms would add overhead for no gain. This branch exists because practitioners routinely salt joins that were never skewed enough to matter, having diagnosed "skew" from a slow job rather than from the key distribution.

Algorithm 1 Skew mitigation decision procedure. Branch structure and thresholds are a direct transcription of `costmodel.decide` and `costmodel.recommend_k` in the released artifact.

Require: H, P, N, M, n, row width w, partitions p, slots C; AQE parameters β (threshold), ϕ (factor), α (advisory size); optional calibrated γ

Ensure: a strategy, and a salt cardinality where one is required

$\rho \leftarrow Hp/N$
if $\rho < \rho_{\min}$ **then**
  return DoNothing {no straggler exists}
end if
$B_{\text{hot}} \leftarrow Hw$; $B_{\text{med}} \leftarrow (N-H)\,w/(p-1)$; $\Theta \leftarrow \max(\beta,\; \phi B_{\text{med}})$
if $B_{\text{hot}} \le \alpha$ **then**
  return Salt {no split possible; the rule is a no-op}
end if
if $B_{\text{hot}} \le \Theta$ **then**
  return Salt {AQE will not classify the partition as skewed}
end if
$\mu \leftarrow Pn/M$
if $\mu \le \mu_{\min}$ **then**
  return AQEOnly
else
  return AQE+Salt
end if

if a salt cardinality is required:
if γ is calibrated **and** the fit is identified **then**
  $k^* \leftarrow \sqrt{\gamma H/P}$
else
  $k^* \leftarrow$ Undefined {omit from the minimum}
end if
return $k_{\text{eff}} \leftarrow \max\big(1, \lfloor \min(k^*, \rho, C) \rceil\big)$

B. Branch 2: AQE will not fire

AQE's rule is conjunctive in *three* conditions, not the two that are usually discussed. The partition must exceed an absolute byte threshold, a multiple of the median, *and* the advisory partition size — because that last is the size the rule splits into, so a hot partition smaller than it cannot be divided at all and the optimisation is a no-op however skewed it is. Section VII finds this third condition to be the binding one in practice, and it is the one practitioners do not tune. A join can be badly skewed in the sense that matters — one task running many times longer than its peers — while sitting below the absolute threshold, whose default is large. In that regime AQE does nothing at all, silently, and the practitioner who has enabled it believes the problem is handled. The remedy is either to salt explicitly or to lower the threshold; the procedure reports which, rather than leaving the user to discover the non-event from the Spark UI.

C. Branch 3: the one-sided/two-sided hinge

This is the substantive branch. When skew is one-sided, splitting the fact-side partition and replicating the small matching probe-side partition to each split is close to free, and AQE resolves the straggler. When both sides carry the same hot key, each of the s splits inherits a large partner. Probe-side work grows with s, and what looked like s-fold relief on the critical path is substantially cancelled.

Detecting which case obtains is cheap, and it is exactly Equation (1): compare P against the probe side's mean rows per key. We use $\mu_{\min} = 2$ throughout, in the text and in the implementation. This is a single group-by on a sample, and it is the highest-value statistic a practitioner can collect before deciding how to fix a slow join — it selects the mechanism *and*, through Equation (4), sets its parameter.

D. Worked example

Take the configuration measured in Section VII: $N = 2\,000\,000$ rows, $p = 16$, $H \approx 445\,743$ rows, $P \approx 11$ rows.

The procedure needs the size of the hot shuffle partition, and here the analytical estimate and the measurement disagree in a way worth stating. Estimating $B_{\text{hot}} = Hw$ gives 7.6 MB; the event log records the largest join-stage shuffle partition at 11.41 MB. The estimate is low because the hot partition also absorbs whatever cold keys hash to it. In the other direction, B_{med} estimates the median at 8.9 MB against a measured 1.35 MB, because it ignores variance among the cold partitions. Hw and $(N-H)w/(p-1)$ are therefore best read as a lower and an upper bound respectively, and where an event log is available the measured sizes should be used instead — as they are in Section VII.

Evaluated on the measured sizes, with $\rho = 3.91$ a straggler exists; the hot partition clears both documented trigger conditions; and it is nonetheless smaller than the advisory split size, so the procedure returns SALT with the reason that no split is possible. Section VII reports what Spark actually did.

VI. Experimental Methodology

A. SKEWBENCH

We implement the study as an open harness rather than a set of scripts, because the artifact is as much the contribution as the numbers. SKEWBENCH comprises a workload generator, six join strategies, an event-log metrics parser, an experiment driver, and the cost model of Section IV. Every result in this paper is produced by a single command and every figure and table is generated from the results file with no manual transcription.

B. Workload generation

The generator produces two Parquet tables with an aerospace engine-telemetry schema, following the NASA C-MAPSS turbofan degradation convention [30] — unit identifier, operational cycle, three operational settings, and N_s sensor channels — so that the synthetic generator can be exchanged for the real dataset without touching the benchmark.

`fact_telemetry` is the skewed side: one row per sensor snapshot, with `engine_id` drawn from a truncated Zipf law over n engines. This mirrors a real fleet, in which a small number of problem units generate a disproportionate share of recorded events. `dim_maintenance` is the probe side: one row per maintenance work order. Its keys are uniform except for the hot key, which carries μ times the mean per-key row count.

The choice to parameterise two-sidedness by μ rather than by drawing the probe side from the same Zipf law is the single most consequential design decision in the harness, and it is not cosmetic. Under a doubly-Zipf design the hot key alone yields $H \times P$ output rows. With $H \approx 4.4 \times 10^5$ and $P \approx 4.4 \times 10^3$ that is roughly 2×10^9 rows from a single key — a thousandfold amplification of a two-million-row fact table, which we observed as a run that made no progress and abandoned. Such a workload does not measure skew mitigation; it measures the cost of materialising a near-cartesian product, and every strategy looks equally bad because all of them are dominated by output volume. Bounding μ keeps the output tractable and promotes P from an artefact of the generator to a swept independent variable, which is what makes the $1/\sqrt{P}$ prediction testable at all.

Skewed variants of TPC-H exist [31], [32] and the harness accepts them, but they do not provide a controllable μ, which is the condition our decision rule turns on. Generation is deterministic given a seed, and the manifest written alongside the data records H and P exactly, so no later stage must re-derive them.

C. Strategies evaluated

1) **baseline** — sort-merge join, AQE disabled, no mitigation.
2) **aqe** — AQE enabled with skew-join optimisation.
3) **salt_uniform**(k) — whole probe side replicated k times, AQE disabled.
4) **salt_selective**(k) — hot-key rows only replicated, AQE disabled.
5) **aqe_salt**(k) — AQE enabled *and* selective salting.
6) **broadcast** — broadcast hash join, where the probe side fits.

Automatic broadcast is disabled throughout (`autoBroadcastJoinThreshold` $= -1$) except in the broadcast arm, so that a strategy under test is never silently replaced by a different plan. Each arm's executed physical plan is captured into the results file, so that the claim "this was a sort-merge join" is auditable rather than asserted.

D. Metrics

We separate metrics by what they license.

Deterministic. Shuffle bytes read and written, records shuffled, and bytes spilled to memory and disk are properties of the physical plan and the partitioning. They are identical on a laptop and on a hundred-node cluster given the same plan and the same data, and the quantitative claims about replication cost rest on them.

Structural. The *skew ratio* — maximum over median task duration within the join stage — and the task-time coefficient of variation are within-run ratios. They cancel most machine-specific scaling while still capturing the straggler that skew produces, and they are how we measure whether a mitigation actually worked as opposed to merely running faster.

Machine-dependent. Wall-clock and executor run time. Reported, used to calibrate γ, and never load-bearing for a transferable claim.

Metrics are recovered by parsing the Spark event log — a JSON-lines file written by Spark itself — rather than through a listener callback. The log survives the driver exiting and can be re-analysed later without re-running anything, which is what makes a result auditable months after the fact. Each measurement is tagged with a unique Spark job group, giving an exact rather than timestamp-inferred mapping from a results row to the tasks that produced it.

E. Execution protocol

Each cell runs one warmup execution, discarded, followed by five timed repetitions. Warmup matters because the first execution of a plan pays JIT compilation, class loading and page-cache misses, and including it inflates the mean unevenly across arms. We report the *median* and interquartile range rather than mean and standard deviation, because run times on a shared machine have a long right tail.

Materialisation is forced with Spark's `noop` sink, so no rows are written to disk or shipped to the driver and the measurement is of the join and its aggregation alone. The terminal operation is a grouped aggregation touching columns from both sides, not a `count()`: counting invites the optimizer to prune the payload and, on some plans, to skip the probe entirely, which would measure column pruning rather than the strategy under test.

F. Platform

Measurements were taken on a single node with 2 cores and 4 GB of driver memory, Apache Spark 3.5.3 on OpenJDK 21, with dedicated local compute — not shared serverless. This choice is deliberate: Nurdin et al. [29] measure roughly twofold run-to-run variance for identical lakehouse queries on shared cloud compute, which would make timing claims meaningless. We are trading absolute scale for measurement control, and Section VIII states what that costs us.

The full configuration grid, the raw event logs, and the exact software versions are included in the artifact.

VII. Results

We report 45 configurations: three probe-side thicknesses ($P \in \{10, 100, 320\}$ rows on the hot key) crossed with six strategies, a salt sweep $k \in \{1, \ldots, 32\}$, and — for the AQE arms — a paired run with partition coalescing disabled. Each cell runs one discarded warmup and five timed repetitions, in randomised order. Every number below is generated from the results file by the released analysis code; none is transcribed by hand.

A. The measurement floor

Selective salting at $k = 1$ compiles to exactly the same physical plan as the unmitigated baseline, so the difference between them is pure measurement noise. We report it first, because it bounds what any other number in this section can mean:

P	10	100	320
Baseline wall (s)	2.6	7.0	18.6
Identical-plan control (s)	3.09	6.97	19.84
Measurement floor	**18.3%**	**0.1%**	**6.5%**

The floor varies sharply with run length, and this governs how the rest of the section may be read. At $P = 100$ the two identical plans agree to 0.1%, and a difference of a few percent is meaningful. At $P = 10$ they differ by 18.3%: those runs last under three seconds, short enough that JVM and scheduler noise dominates, and **no claim can be supported at that probe thickness at all**. We report the $P = 10$ column throughout for completeness and draw no conclusions from it.

Two points about how this number is produced. All 45 cells ran in a single Spark application: an earlier attempt gap-filled four cells from a second application, and those cells were systematically slower and drifting within their own run, which inflated this control from 2% to 29%. Wall-clock comparisons across Spark applications measure JVM warmth, not plans, and the harness now records the application id and refuses to compare across it. Separately, an earlier version executed cells in configuration order, placing the baseline first on a cold JVM; that inflated the same control to 17.7% and, with it, every speedup measured against that baseline. Cell order is now randomised under a fixed seed.

B. The baseline is severely skewed

Without mitigation, the join stage's longest task takes 1.2 s, 5.6 s and 17.0 s at the three probe thicknesses, against median tasks of a fraction of a second. At $P = 320$ a single task runs roughly two orders of magnitude longer than its median peer. This is a textbook straggler, and no amount of additional compute would remove it.

C. AQE did not help at any probe thickness

This is the paper's central measurement, and it is a null result.

P	10	100	320
Baseline wall (s)	2.6	7.0	18.6
AQE wall (s)	2.9	6.9	18.3
AQE speedup	**0.91×**	**1.01×**	**1.02×**
Baseline longest task (s)	1.2	5.6	17.0
AQE longest task (s)	1.8	6.2	17.5

At $P = 100$, where the measurement floor is 0.1%, AQE returns 1.01×. At $P = 320$, floor 6.5%, it returns 1.02× — inside the floor. And the row that settles it is the last one: enabling Adaptive Query Execution on a join whose longest

task ran 17.0 s produced a longest task of 17.5 s. **The critical path is not shortened at any probe thickness; in all three it is marginally longer.**

We are careful about what this does and does not say. It is not a claim that AQE is useless in general — it does other useful things, and one of them shows up below. It is a claim that on *this* join, with Spark's skew machinery enabled and a straggler two orders of magnitude out, AQE delivered no improvement distinguishable from measurement noise, and left the straggler itself untouched. The next two subsections establish why, and the answer is specific enough to act on.

D. The benefit that does exist is coalescing, not skew handling

AQE bundles several rewrites. To separate them we ran every AQE cell twice, identical but for `spark.sql.adaptive.coalescePartitions.enabled`:

P	10	100	320
AQE, coalescing on (s)	2.88	6.90	18.33
AQE, coalescing off (s)	3.39	7.50	18.38
Baseline (s)	2.61	6.96	18.62
Longest task, coal. off (s)	1.3	5.7	16.9

With coalescing disabled, AQE is indistinguishable from no AQE at all. The longest task is unchanged in every column. Whatever small benefit AQE provided came from merging small post-shuffle partitions — which lowers fixed per-task overhead and does precisely nothing for a straggler.

E. The skew-join rule never fired, and why

Reading the *final* adaptive plan for every AQE cell explains the result directly. Capturing it requires care: executing into a `noop` sink builds a separate query execution and leaves the inspected plan reporting `isFinalPlan=false`, showing the pre-adaptive shape and hiding exactly the rewrite one wants to audit. Read after an action on the DataFrame itself, every plan contains `AQEShuffleRead coalesced` and never `AQEShuffleRead skewed`. **Spark's skew-join rule did not fire.**

The reason is a precondition that is documented but almost never discussed. A skewed partition is split into pieces of roughly `advisoryPartitionSizeInBytes`; if the hot partition is *smaller* than the advisory size, there is no split to make and the rule is a no-op however skewed the partition is. The measured hot shuffle partition here is 11.41 MB against a 16-partition advisory size of 16 MB.

This is a claim about a non-event, so it needs a positive control. We swept the two documented trigger conditions to their most permissive settings — `skewedPartitionThresholdInBytes` from 64 MB down to 256 KB, and `skewedPartitionFactor` from 5 down to 1. The rule still never fired. Lowering only the advisory size engaged it immediately:

Partitions	Advisory	Threshold	Skew-join applied
16	16 MB	8 MB	no
16	1 MB	1 MB	**yes**
64	1 MB	1 MB	**yes**
200	256 KB	256 KB	**yes**

One configuration change, no code change. We regard this as the most practically consequential finding in the paper: practitioners tune the two parameters whose names contain the word *skew*, and the parameter that actually gates the optimisation at moderate data volumes is the one that does not. Section V's procedure encodes all three preconditions for this reason, and predicted the non-event from data statistics alone.

F. Salting worked, and its advantage grows with the probe side

P	10	100	320
Best selective-salt k	2	2	2
Salting speedup	0.97×	1.34×	1.68×
Longest task, baseline (s)	1.2	5.6	17.0
Longest task, salted (s)	0.8	3.2	9.1
Critical-path cut	**1.6**×	**1.7**×	**1.9**×

At $P = 10$ the result is inside the 18.3% floor and we draw no conclusion. At $P = 100$ and $P = 320$, where the floor is 0.1% and 6.5%, salting returns 1.34× and 1.68× — comfortably outside noise — and cuts the critical path by 1.7× and 1.9× where AQE cut it by nothing.

One nuance worth recording, because it bears on the cost model. The k that minimises *wall-clock* is 2 at $P = 320$, but a larger k minimises the *critical path*: at $k = 16$ the longest task falls further still. The two optima differ because on two task slots the extra shuffle from a larger k costs more than the additional straggler relief returns. On a cluster with more slots they should converge, and that is one of the things the released cluster configuration is designed to check.

G. Broadcast dominates at this scale, and we say so

The broadcast arm completes in 2.62 s, 5.60 s and 13.32 s — faster than every shuffle-join strategy at every probe thickness. This is expected and it is worth stating plainly rather than omitting: our dimension table is well under Spark's default broadcast threshold, and a practitioner in that regime should broadcast and stop reading. Automatic broadcast is disabled throughout precisely so that the shuffle-join behaviour we set out to study is not silently replaced. The results here apply to joins whose dimension side exceeds the broadcast threshold, which is the regime in which skew mitigation is a live question at all.

H. Replication cost matches the model exactly

Shuffle volume is a property of the plan and the data rather than of the machine, so it is the measurement that transfers off a single node. Measured in shuffle *records* it does more than transfer: it matches Equation (3) to the row.

An earlier version of this analysis measured replication in shuffle *megabytes* and reported a sevenfold gap between the two salting variants. That was wrong, and the error is worth recording because it is easy to repeat. Selective salting replicates only $P(k-1)$ rows — a few hundred across the entire sweep — while the megabyte growth was dominated by the one-column `_salt` addition to every one of two million fact rows. Over 99% of what was attributed to replication was the extra column. Records have no such confound: they are exactly the quantity the model's $bP(k-1)$ term refers to.

Fitting a least-squares line to shuffle records against k over a matched range gives:

Variant	records per unit k	model predicts	R^2
Selective salting	11	$P = 11$	1.000
Uniform salting	20000	$M = 20\,000$	1.000

Selective salting adds 11 rows per salt value, and the hot key has 11 rows on the probe side. Uniform salting adds 20000 rows per salt value, and the probe table has 20 000 rows. The measured ratio, 1818, equals M/P exactly, which is what the model predicts, and both fits are linear to $R^2 = 1.000$.

This is the paper's cleanest quantitative result, and we would emphasise *why* it is clean rather than merely report it. Records are counted, not timed. They do not vary between repetitions, they do not depend on the machine, and they are not confounded by anything the scheduler does. The replication term of the cost model is therefore not an approximation fitted to noisy observations — it is exact, and the distinction between the two salting variants is a factor of M/P, which for any realistic fact–dimension join is three orders of magnitude, not seven-fold.

The practical reading is that **the tutorial formulation of salting is the expensive one**, and by a margin far wider than the folklore suggests.

I. Where the cost model held, and where it did not

The model's structural claims are supported. The replication term is exact, as above. Salting monotonically reduces the critical path: at the thickest probe side the longest join-stage task falls from 18.6 s unmitigated to a small fraction of that under salting.

The *convexity* of $T(k)$ in wall-clock, however, did not resolve, and we report that plainly rather than presenting a fitted curve as a confirmation.

Fitting Equation (3) to the selective-salting sweeps gives R^2 of 0.312, 0.288 and 0.820. Those look respectable and are misleading. The quantity that matters is whether the replication coefficient b — the entire denominator of k^* — is resolved by the data at all:

TABLE I
BEST CONFIGURATION PER STRATEGY, WITH SPEEDUP OVER THE UNMITIGATED BASELINE. μ IS THE PROBE-SIDE HOT-KEY MULTIPLIER; SKEW IS THE JOIN-STAGE MAXIMUM-TO-MEDIAN TASK DURATION RATIO.

μ	strategy	k	wall (s)	speedup	skew
1.00	baseline	1	2.61	1.00	7.48
1.00	broadcast	1	2.62	1.00	1.05
1.00	salt_selective	2	2.69	0.97	5.82
1.00	salt_uniform	2	2.78	0.94	4.97
1.00	aqe	1	2.88	0.91	1.02
1.00	aqe_salt	8	3.01	0.87	2.50
10.00	salt_selective	2	5.19	1.34	23.87
10.00	aqe_salt	8	5.53	1.26	1.32
10.00	broadcast	1	5.60	1.24	1.03
10.00	salt_uniform	8	5.68	1.23	6.39
10.00	aqe	1	6.90	1.01	3.54
10.00	baseline	1	6.96	1.00	35.97
32.00	salt_selective	2	11.08	1.68	68.38
32.00	aqe_salt	8	11.67	1.60	13.91
32.00	salt_uniform	8	11.72	1.59	17.45
32.00	broadcast	1	13.32	1.40	1.03
32.00	aqe	1	18.33	1.02	10.39
32.00	baseline	1	18.62	1.00	101.01

P	10	100	320
R^2	0.312	0.288	0.820
$\lvert b\rvert/\mathrm{se}(b)$	1.16	0.06	1.14
b identified?	no	no	no
k^*	not identified	not identified	not identified

At two of three workloads b is statistically indistinguishable from zero. A k^* computed from such a fit is an artefact of noise, and our implementation refuses to return one: `optimal_k_unconstrained` yields NAN rather than a number, and the recommendation falls back to the saturation and parallelism ceilings alone. Only 0 of 3 fits supports a cost-optimum at all.

A second, independent sign of trouble: the fitted $\gamma = a/b$ varies by $121\times$ across three workloads that differ only in P, on one machine, in one session. Section IV advances γ-stability as the property that would make the closed form practically useful. Our own data contradict it.

We therefore make no claim that the closed form is validated. What we can say is precise: the replication *term* is exact; the replication *cost in time* is, at this scale, too small for the convexity it implies to be observable; and the conditions under which it should become observable — cluster-scale network shuffle, or a substantially thicker probe side — are identified and configured in the released artifact.

The practical consequence points the opposite way from the folklore, and is worth stating for practitioners rather than only for reviewers. **With selective salting, over-provisioning k is close to free.** The warning against large k is well founded for the uniform variant, where the replication cost is M/P times larger and clearly measurable. For the selective variant, the sensible advice is to salt at the saturation bound and stop worrying about the exact optimum.

J. Summary

- AQE's advantage over an unmitigated baseline fell from $0.91\times$ to $1.02\times$ as the probe-side hot key grew from 10 to 320 rows, while salting's rose from $0.97\times$ to $1.68\times$.
- Spark's skew-join rule never fired on a join with a 101:1 task-time skew ratio, verified from final adaptive plans. The measured AQE benefit was partition coalescing.
- Neither documented trigger knob was responsible; the binding condition was `advisoryPartitionSizeInBytes` exceeding the hot partition. Lowering it engaged the rule immediately.
- Selective salting replicates 11 rows per salt value against uniform salting's 20000 — a ratio of 1818, exactly the M/P the cost model predicts, with both fits linear to $R^2 = 1.000$.
- The replication term of the cost model is confirmed exactly. Its consequence for runtime is not: the coefficient b is statistically unidentified at two of three workloads, fitted γ varies by $121\times$ on one machine, and we therefore do not claim the closed form for k^* is validated.

VIII. Threats to Validity

A. Single-node measurement

This is the principal threat and we state it without hedging. All measurements were taken on one machine with a small number of cores. Absolute wall-clock times therefore do not transfer to a cluster, and we make no claim that they do.

What survives the objection is specific. Shuffle volume and spill volume are determined by the physical plan and the data partitioning, not by the machine: selective salting with $k = 32$ replicates the same probe-side rows on a laptop as on a hundred nodes. The skew ratio is a within-run ratio of task durations and cancels most machine-specific scaling. And the model's structure — convexity in k, the $\sqrt{H/P}$ dependence, the two ceilings — is analytic, not measured; the experiments test whether it holds, and the calibration constant γ is expected to differ per platform, which is why it is a calibrated parameter and not a published number.

What does *not* survive is any claim about network shuffle. A single-node Spark shuffle writes to local disk and reads back; a cluster shuffle crosses the network, where the replication cost of salting is materially higher. Our estimate of b is therefore *conservative*, and the true k^* on a cluster should be *smaller* than we report — which strengthens rather than weakens the paper's central practical claim that customary salt cardinalities are too large. We flag this asymmetry explicitly so that a reader does not have to infer it.

The parallelism ceiling is the sharpest limitation. With few task slots, C binds in Equation (7) for most configurations, so our measurements exercise that branch more than the cost-optimum branch. We address this by reporting the model's projection across C rather than only the measured point, and by being clear that the projection is a projection.

B. Synthetic data

Zipf is a model of skew, not skew itself. Real key distributions have structure — correlated hot keys, temporal drift, keys hot on one side only in certain periods — that a stationary Zipf law does not reproduce. We mitigate this by using an established distributional form, by sweeping θ rather than fixing it, by adopting a schema from a real prognostics dataset [30], and by including a multi-hot-key configuration. The harness accepts skewed TPC-H [31] and real C-MAPSS data as drop-in replacements. But a study on production key distributions would be stronger, and we do not have one.

C. Bounded probe-side multiplicity

Our two-sided workloads cap the hot key's probe-side representation at μ times the mean rather than letting it follow the fact side's Zipf law. This is a deliberate bound, justified in Section VI, but it is a restriction on generality: production joins do exist in which both sides are severely concentrated and the output genuinely explodes. For those, the binding problem is output cardinality rather than straggler relief, and neither mechanism studied here is the right answer — the correct response is to change the query, not to tune the join. We regard that as a different problem, and we do not claim our results speak to it.

D. Version specificity

AQE's skew-join rule, its defaults, and the interaction between `coalescePartitions` and `skewJoin` all change between Spark releases, and the production descendant described by Xue et al. [10] differs from the open-source rule. Our measurements pin one version, recorded in the artifact. The cost model does not depend on the version; the decision procedure's Branch 2, which reasons about the trigger condition, does.

E. Construct validity of the skew ratio

Maximum over median task duration is a proxy, and it has a confound serious enough that we no longer use it as a primary metric.

The problem is the denominator. AQE's partition coalescing merges the sixteen shuffle partitions into three, which changes what "the median task" refers to without changing the straggler at all. Comparing a three-task stage's max-over-median against a sixteen-task stage's is comparing different quantities. In our data this inverts the ranking: at the thickest probe side the AQE arm reports a far lower skew ratio than the unmitigated baseline while its slowest task is *longer* in absolute terms. Any reader who took the ratio at face value would conclude the opposite of the truth.

We therefore report three things instead. The *absolute maximum task duration* in the join stage, which is the critical path and has no denominator to move. The *straggler index*, maximum over *mean* task duration: because total task time is roughly conserved under coalescing, the mean is stable where the median is not. And *task counts* alongside every ratio, so that a reader can see immediately when two numbers are not

comparable. The max-over-median ratio is retained only for continuity with the practitioner literature, which universally quotes it.

The residual confounds remain: a near-zero median makes the ratio unstable, and task-duration variance from scheduling and garbage collection is not skew. Both are reasons to lead with shuffle-byte and record distributions, which have no such confound.

F. The dimension side is small enough to broadcast

Our probe table is well under Spark's default ten-megabyte automatic broadcast threshold. In a default configuration this join would be planned as a broadcast hash join, which has no shuffle and therefore no shuffle skew, and neither mechanism we study would be engaged at all. We disable automatic broadcast throughout precisely to isolate the shuffle-join behaviour we set out to measure, and we run the broadcast arm so that its dominance at this scale is visible in the results rather than hidden by the configuration.

The honest scope statement follows: our results apply to fact–dimension joins in which the dimension side exceeds the broadcast threshold, which is the regime where skew mitigation is a live question. A reader whose dimension table fits in a broadcast should broadcast it and stop reading.

G. Execution order and the noise floor

An earlier version of this harness executed cells in configuration order, which placed the unmitigated baseline first in every workload block — on a cold JVM, and therefore systematically slow. Because every speedup is measured against that baseline, the bias inflated all of them.

We detected this with a control the harness generates for free: selective salting at $k = 1$ compiles to exactly the same physical plan as the baseline, so any difference between the two is measurement noise. The discrepancy reached 17.7% on the first workload while remaining under 0.3% on the later ones — a clear order effect rather than random variation. Cell order is now randomised under a fixed seed, a session-level warmup precedes the grid, and the control is computed and reported for every run. It is included in the artifact.

We state the consequence rather than burying it: **no speedup smaller than the control discrepancy should be read as a result**, in this paper or in any run of this harness.

H. Statistical power

Five repetitions per cell, and three for the configuration probes, is few. We report medians and interquartile ranges rather than means and standard deviations, and we do not attach p-values to five-against-five comparisons, where the smallest attainable two-sided rank-test p is above the conventional threshold in any case. Differences of a few percent in wall-clock are within noise at this sample size and we do not interpret them. The deterministic counters — shuffle bytes and records — carry the claims that need precision, and they do not vary between repetitions at all.

I. One partition count, one core count

Every reported cell uses sixteen shuffle partitions and two task slots. The parallelism ceiling therefore binds the recommendation in essentially every configuration, which means the cost-optimum branch of Equation (7) — the branch the closed form exists to serve — is never the operative one in our own data. This is the sharpest reason the cluster validation described in Section IX matters, and it is why we release the cluster configuration alongside the single-node one.

J. Generalisation beyond inner equi-joins

We evaluate inner equi-joins on a single key. Outer joins, multi-key joins, joins whose skewed key is also the aggregation key, and joins feeding a subsequent shuffle all plausibly behave differently. The model's derivation does not obviously break for these cases, but we have not tested them, and we do not claim them.

K. Absence claims

Section II rests on two negative results: no independent evaluation of Spark's skew-join rule, and no peer-reviewed treatment of salting. Negative results in literature search are only as good as the search. Ours covered dblp, the arXiv full-metadata API, Crossref, and domain-scoped searches across the major publishers, but Semantic Scholar and OpenAlex were unavailable, and at least one relevant paper [23] is not indexed in dblp. We therefore present these as strong evidence of a gap rather than as proof of one, and we cite the nearest existing work explicitly rather than claiming an empty field.

IX. Conclusion

Spark ships an automatic remedy for skewed joins, and practitioners continue to salt by hand. We set out to determine which they should use, and how much salt to use when salting. The answer on our workload was blunter than expected: Spark's skew-join rule never engaged at all, so there was nothing to choose between.

The measurement is a null result and we report it as one. Enabling AQE on a join whose longest task ran 17.0 s produced a longest task of 17.5 s, with speedups of 1.01× and 1.02× against measurement floors of 0.1% and 6.5%. A paired control with coalescing disabled shows that what little AQE contributed was partition merging, which lowers fixed overhead and does nothing for a straggler. Selective salting, on the same workload, cut the critical path 1.9×.

The cause is a precondition that is documented and almost never discussed. A skewed partition is split into pieces of `advisoryPartitionSizeInBytes`; a hot partition smaller than that size cannot be split at all, and the rule becomes a no-op however extreme the skew. Our hot partition measured 11.41 MB against a 16 MB advisory size. Sweeping the two parameters whose names contain the word *skew* to their most permissive settings changed nothing; lowering the advisory size engaged the rule immediately. Practitioners tune the two knobs that are named for the problem, and the one

that gates the solution at moderate data volumes is not among them.

On the cost model we are more circumspect than we expected to be. The replication term is confirmed exactly in shuffle records — 11 rows per salt value for selective salting against 20000 for uniform, a ratio of 1818 matching the predicted M/P with $R^2 = 1.000$. Its consequence for runtime is not confirmed: the replication coefficient is statistically unidentified in all 3 fits, and in 1 of them it comes out with the wrong sign. At this scale replication cost is not merely unresolved in wall-clock; it is absent. We therefore present the closed form $k^* = \sqrt{\gamma H/P}$ as theory, state plainly that this study does not validate it, and release the cluster configuration under which the prediction — that network shuffle raises the coefficient and lowers k^* — becomes testable.

The decision between mechanisms turns on a statistic that costs one sampled group-by to obtain. When skew is one-sided and above AQE's trigger, AQE alone suffices and manual salting adds replication for little gain. When the same key is heavy on both sides, split-and-replicate degenerates and salting remains necessary. When the hot partition sits below AQE's absolute threshold, the rule does nothing at all — silently — and a practitioner who has enabled AQE believes the problem is handled.

The artifact matters as much as the result. We cannot outscale the organisations whose systems papers describe this machinery at production volume, and we do not try. What a practitioner-built harness can contribute is specification: a public, reproducible generator with a declared skew taxonomy, a metrics layer that separates deterministic counters from machine-dependent timings, and an honest account of what a single node does and does not license. SKEWBENCH is released under the MIT licence.

Three directions follow. The most useful is validation at cluster scale, where the network shuffle makes b larger and k^* correspondingly smaller than we report; our prediction is directional and testable. The second is to fold γ estimation into the engine, so that k^* is computed rather than configured — salting is currently the one skew remedy Spark cannot apply for you, and the model here is simple enough to be a planner rule. The third is to extend the taxonomy: multi-key joins, outer joins, and skew that drifts over time are all common in production and none is covered here.

Artifact Availability

The SKEWBENCH harness, the workload generator, the raw event logs, the analysis pipeline, and the scripts that produce every figure and table in this paper are released under the MIT licence and archived with a DOI. All results reported here are reproducible with a single command on a commodity laptop.

References

[1] C. B. Walton, A. G. Dale, and R. M. Jenevein, "A taxonomy and performance model of data skew effects in parallel joins," in *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*. Morgan Kaufmann, 1991, pp. 537–548. [Online]. Available: http://www.vldb.org/conf/1991/P537.PDF

[2] D. J. DeWitt, J. F. Naughton, D. A. Schneider, and S. Seshadri, "Practical skew handling in parallel joins," in *Proceedings of the 18th International Conference on Very Large Data Bases (VLDB '92)*. Morgan Kaufmann, 1992, pp. 27–40. [Online]. Available: http://www.vldb.org/conf/1992/P027.PDF

[3] R. Chaiken, B. Jenkins, P.-Å. Larson, B. Ramsey, D. Shakib, S. Weaver, and J. Zhou, "SCOPE: Easy and efficient parallel processing of massive data sets," *Proceedings of the VLDB Endowment*, vol. 1, no. 2, pp. 1265–1276, 2008.

[4] G. Ananthanarayanan, S. Kandula, A. Greenberg, I. Stoica, Y. Lu, B. Saha, and E. Harris, "Reining in the outliers in Map-Reduce clusters using Mantri," in *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10)*. USENIX Association, 2010, pp. 265–278. [Online]. Available: http://www.usenix.org/events/osdi10/tech/full_papers/Ananthanarayanan.pdf

[5] Y. Kwon, M. Balazinska, B. Howe, and J. Rolia, "SkewTune: Mitigating skew in MapReduce applications," in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12)*. ACM, 2012, pp. 25–36.

[6] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*. San Jose, CA, USA: USENIX Association, 2012, pp. 15–28. [Online]. Available: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

[7] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational data processing in Spark," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD '15)*. ACM, 2015, pp. 1383–1394.

[8] W. Fan, H. van Hövell, and M. Xue, "Adaptive query execution: Speeding up Spark SQL at runtime," Databricks Engineering Blog, https://www.databricks.com/blog/2020/05/29/adaptive-query-execution-speeding-up-spark-sql-at-runtime.html, May 2020, published 29 May 2020. Accessed 18 August 2026.

[9] Apache Spark, "Performance tuning — Spark SQL guide: Adaptive query execution and optimizing skew join," https://spark.apache.org/docs/latest/sql-performance-tuning.html, apache Spark official documentation. Accessed 18 August 2026.

[10] M. Xue, Y. Bu, A. Somani, W. Fan, Z. Liu, S. Chen, H. van Hovell, B. Samwel, M. Mokhtar, R. Korlapati, A. Lam, Y. Ma, V. Ercegovac, J. Li, A. Behm, Y. Li, X. Li, S. Krishnamurthy, A. Shukla, M. Petropoulos, S. Paranjpye, R. Xin, and M. Zaharia, "Adaptive and robust query execution for lakehouses at scale," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3947–3959, 2024.

[11] M. A. Irandoost, A. M. Rahmani, and S. Setayeshi, "MapReduce data skewness handling: A systematic literature review," *International Journal of Parallel Programming*, vol. 47, no. 5–6, pp. 907–950, 2019.

[12] B. Gufler, N. Augsten, A. Reiser, and A. Kemper, "Handling data skew in MapReduce," in *Proceedings of the 1st International Conference on Cloud Computing and Services Science (CLOSER 2011)*. SciTePress, 2011, pp. 574–583.

[13] J. Myung, J. Shim, J. Yeon, and S.-g. Lee, "Handling data skew in join algorithms using MapReduce," *Expert Systems with Applications*, vol. 51, pp. 286–299, 2016.

[14] Y. Shen, J. Xiong, and D. Jiang, "SrSpark: Skew-resilient Spark based on adaptive parallel processing," in *Proceedings of the 26th IEEE International Conference on Parallel and Distributed Systems (ICPADS 2020)*. IEEE, 2020, pp. 466–475.

[15] Z. Tang, W. Lv, K. Li, and K. Li, "An intermediate data partition algorithm for skew mitigation in Spark computing environment," *IEEE Transactions on Cloud Computing*, vol. 9, no. 2, pp. 461–474, 2021.

[16] Y. Kwon, M. Balazinska, B. Howe, and J. Rolia, "SkewTune in action: Mitigating skew in MapReduce applications," *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 1934–1937, 2012.

[17] A. Metwally, "Scaling and load-balancing equi-joins," *ACM Transactions on Database Systems*, vol. 50, no. 2, pp. 5:1–5:41, 2025, introduces the AM-Join algorithm; preprint first posted 2022.

[18] J. He, Y. Cui, C. Li, J. Jiang, Y. Hou, and H. Chen, "AQORA: A learned adaptive query optimizer for Spark SQL," 2025, title of arXiv v1; later revised to "AQORA: A Fast Learned Adaptive Query Optimizer with Stage-Level Feedback for Spark SQL" (v2) and "LQRS: Learned Query Re-optimization Framework for Spark SQL" (v3).

[19] J. Xin, K. Hwang, and Z. Yu, "LOCAT: Low-overhead online configuration auto-tuning of Spark SQL applications," in *Proceedings of the 2022 ACM SIGMOD International Conference on Management of Data (SIGMOD '22)*. ACM, 2022, pp. 674–684.

[20] C. Lyu, Q. Fan, P. Guyard, and Y. Diao, "A Spark optimizer for adaptive, fine-grained parameter tuning," *Proceedings of the VLDB Endowment*, vol. 17, no. 11, pp. 3565–3579, 2024.

[21] R. Suri, I. Gofman, G. Yu, and J. C. Cresswell, "Zero-execution retrieval-augmented configuration tuning of Spark applications," 2025.

[22] Y. Zhang, Y. Chronis, J. M. Patel, and T. Rekatsinas, "Simple adaptive query processing vs. learned query optimizers: Observations and analysis," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2962–2975, 2023.

[23] A.-C. Phan, T.-C. Phan, H.-P. Cao, and T.-N. Trieu, "Comparative analysis of skew-join strategies for large-scale datasets with MapReduce and Spark," *Applied Sciences*, vol. 12, no. 13, p. 6554, 2022.

[24] T. Siddiqui, A. Jindal, S. Qiao, H. Patel, and W. Le, "Cost models for big data query processing: Learning, retrofitting, and our findings," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. ACM, 2020, pp. 99–113.

[25] J. Zhou, P.-Å. Larson, and R. Chaiken, "Incorporating partitioning and parallel plans into the SCOPE optimizer," in *Proceedings of the 26th IEEE International Conference on Data Engineering (ICDE 2010)*. IEEE, 2010, pp. 1060–1071.

[26] V. Leis and M. Kuschewski, "Towards cost-optimal query processing in the cloud," *Proceedings of the VLDB Endowment*, vol. 14, no. 9, pp. 1606–1612, 2021.

[27] M. Shen, Y. Zhou, and C. Singh, "Magnet: Push-based shuffle service for large-scale data processing," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3382–3395, 2020.

[28] J. Camacho-Rodríguez, A. Agrawal, A. Gruenheid, A. Gosalia, C. Petculescu, J. Aguilar-Saborit, A. Floratou, C. Curino, and R. Ramakrishnan, "LST-Bench: Benchmarking log-structured tables in the cloud," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 59:1–59:26, 2024.

[29] J. Nurdin, W. Liu, R. McCreadie, and L. Thamsen, "Predicting lakehouse performance in clouds: An empirical exploration of query runtime variance," 2026, to appear in the Proceedings of the 19th IEEE International Conference on Cloud Computing (CLOUD).

[30] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proceedings of the 2008 International Conference on Prognostics and Health Management (PHM 2008)*. Denver, CO, USA: IEEE, 2008, pp. 1–9.

[31] Microsoft, "Program for TPC-H data generation with skew," https://www.microsoft.com/en-us/download/details.aspx?id=52430, microsoft Download Center; `TPCDSkew.zip`. Accessed 18 August 2026.

[32] Transaction Processing Performance Council, "TPC Benchmark™ H (decision support) standard specification, revision 3.0.1," https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf, 2022, revision date 28 April 2022. Accessed 18 August 2026.